



School of Electrical & Electronic Engineering

Introduction to Cobot

Robotics and Smart Automation Innovation Hub
(S1-B3c-28)

Cobot User Manual

Prepared by: Dawson Eng & Li Jiang Peng

Version history	Release	Changes
1.0	30 May 25	Initial Release
2.0	01 July 25	Added Experimental tasks

Table of Contents

Table of Figures	4
Fairino FR3 Collaborative Robot	6
Excellent Performance Efficiency First	6
Awesome Security Comprehensive Protection.....	6
Flexible Deployment No Entry Threshold	6
Efficient Protection Stable & Reliable	7
Complete Ecosystem Convenient Integration	7
Cobot wiring diagram	7
Cobot Operation Flow	9
User Configurable Settings	14
Program command	15
Logic Command Interface	16
While command	16
If ... else command	17
Goto command	18
Wait command	19
Pause command	23
Dofile command.....	24
Var command.....	25
Motion command interface.....	27
PTP command	27
Experimental Tasks	29
A. Cobot arm movement with electrical gripper utilization via software	29
Necessary Skills.....	29
Lua Programming.....	29
B. Cobot arm movement with electrical gripper utilization via communication protocol.....	33
Necessary Skills.....	33
Lua Programming.....	33
Python Programming.....	33
Communication Protocols	34

RS-485 Communication Protocol.....	41
Modbus RTU Communication Protocol.....	44

Table of Figures

Figure 1: FR3 Cobot.....	6
Figure 2: Cobot wiring diagram	7
Figure 3: Operational Flow Step One.....	9
Figure 4: Operational Flow Step One Code	9
Figure 5: Operational Flow Step Two	10
Figure 6: Operational Flow Step Two Code	10
Figure 7: Operational Flow Step Three.....	11
Figure 8: Operational Flow Step Three Code	11
Figure 9: Operational Flow Step Four	12
Figure 10: Operational Flow Step Four Code.....	12
Figure 11: Operational Flow Step Five	13
Figure 12: Programming Interface – GUI	14
Figure 13: Programming Interface – Lua	14
Figure 14: Logic Command Interface	16
Figure 15: if. . . else Command Interface	18
Figure 16: Goto Command Interface.....	19
Figure 17: WaitTime Command Interface	20
Figure 18: WaitDI Command Interface	21
Figure 19: WaitMultiDI Command Interface.....	22
Figure 20: WaitAI Command Interface	23
Figure 21: Pause Command Interface	24
Figure 22: Dofile Comand Interface	25
Figure 23: Var Command Interface	26
Figure 24: Motion Command Interface.....	27
Figure 25: PTP Command Interface.....	28
Figure 26: Lua Codes 1	30
Figure 27: Lua Codes 2	30
Figure 28: Lua Codes 3	30
Figure 29: Lua Codes 4	30
Figure 30: Lua Codes 5	31
Figure 31: Lua Codes 6	31
Figure 32: Lua Codes 7	31
Figure 33: Lua Codes 8	32
Figure 34: Lua Codes 9	32
Figure 35: Python Example Code 1	34

Figure 36: Python Example Code 2	34
Figure 37: TCP Server.....	37
Figure 38: TCP Client	37
Figure 39: UDP Server	39
Figure 40: UDP Client	39
Figure 41: HTTP Server.....	41
Figure 42: HTTP Client	41
Figure 43: RS-485 Communication 1	42
Figure 44: RS-485 Communication 2	42
Figure 45: RS-485 Communication 3	42
Figure 46: RS-485 Communication 4	43
Figure 47: RS-485 Communication 5	43
Figure 48: RS-485 Communication 6	44
Figure 49: Modbus RTU Communication 1	45
Figure 50: Modbus RTU Communication 2\	46
Figure 51: Modbus RTU Communication 3	46
Figure 52: Modbus RTU Communication 4	47
Figure 53: Modbus RTU Communication 5	47

Fairino FR3 Collaborative Robot



Figure 1: FR3 Cobot

FR3 is a new generation of six-axis cobot product based on the definition of human-robot collaboration technology. Its scenarios cover 3C, semiconductor, metal processing, industrial gluing, new retail and other industries, creating the optimal solution for flexible intellectual manufacturing applications.

Excellent Performance Efficiency First

The system adopts the leading EtherCAT bus architecture, with strong real-time and high stability, which comprehensively improves the production performance in various scenarios.

Awesome Security Comprehensive Protection

The core components are self-developed and self-produced, with patented design scheme, modular design scheme, and full closed-loop joint torque feedback to ensure safety and efficiency.

Flexible Deployment No Entry Threshold

Adopt graphical programming, smooth drag teaching function, multi-axis linkage, easy integration.

Efficient Protection Stable & Reliable

IP66 protection, perfectly adapted to all types of application scenarios, safer and more reliable.

Complete Ecosystem Convenient Integration

Support ROS secondary development, self-developed open platform, to meet more diverse needs, support a variety of communication protocols. Provide convenience for robot integrators.

Cobot wiring diagram



Figure 2: Cobot wiring diagram

It illustrates the wiring connections for setting up a Fairino collaborative robot (cobot) system. The diagram is divided into different sections showing how each component connects to the Control Box, which is the central hub of the system.

1. Robot Body ↔ Control Box

- Connection: The Robot Body connects to the Control Box via a Robot Connecting Cable.

- Purpose: This cable transmits power and control signals to the robot's joints and sensors.

2. Robot Connecting Cable

- Shown separately to emphasize both ends of the cable.
- One end plugs into the robot base, and the other end into the control box.
- Purpose: Facilitates communication and power between the Control Box and Robot Body.

3. Safety Box & Teach Pendant ↔ Control Box

- Safety Box: Includes an emergency stop button for safety.
- Teach Pendant: Used to manually control or program the robot.
- Connection: Plugged into the control box through a dedicated port.

4. Power Cord ↔ Control Box

- Connection: Supplies electrical power from a wall outlet to the Control Box.
- Purpose: Powers the entire cobot system.

Cobot Operation Flow

This section outlines a use case demonstrating the operational sequence of the cobot, as depicted in the accompanying video. The scenario involves picking up a bottle from one corner of a table and relocating it to another corner.

1. Initialization: Begin by resetting the manipulator to its initial position.

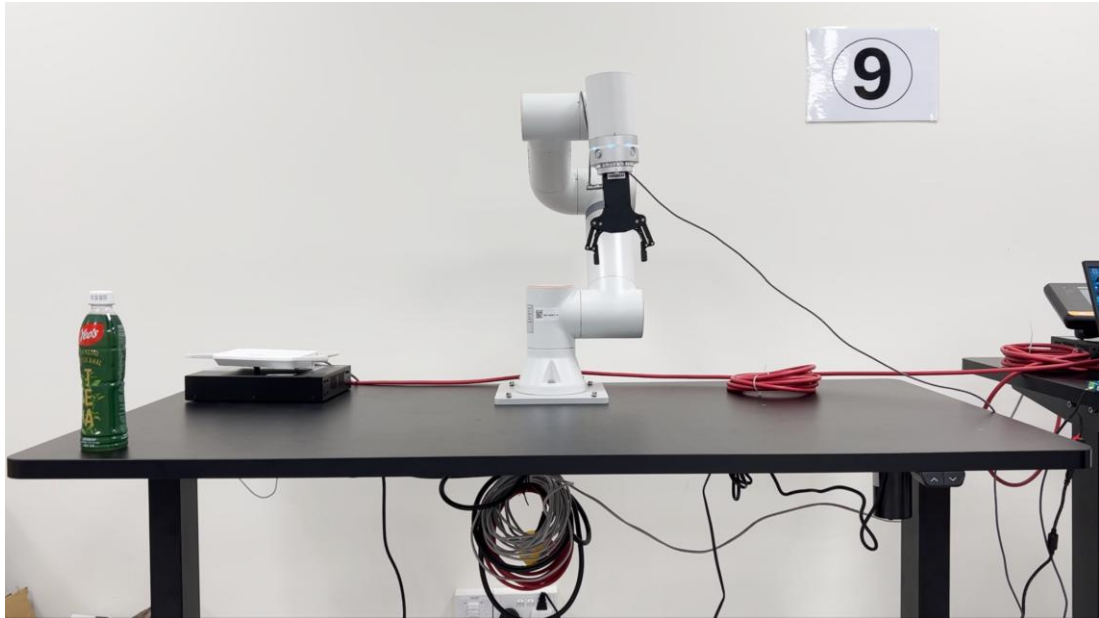


Figure 3: Operational Flow Step One

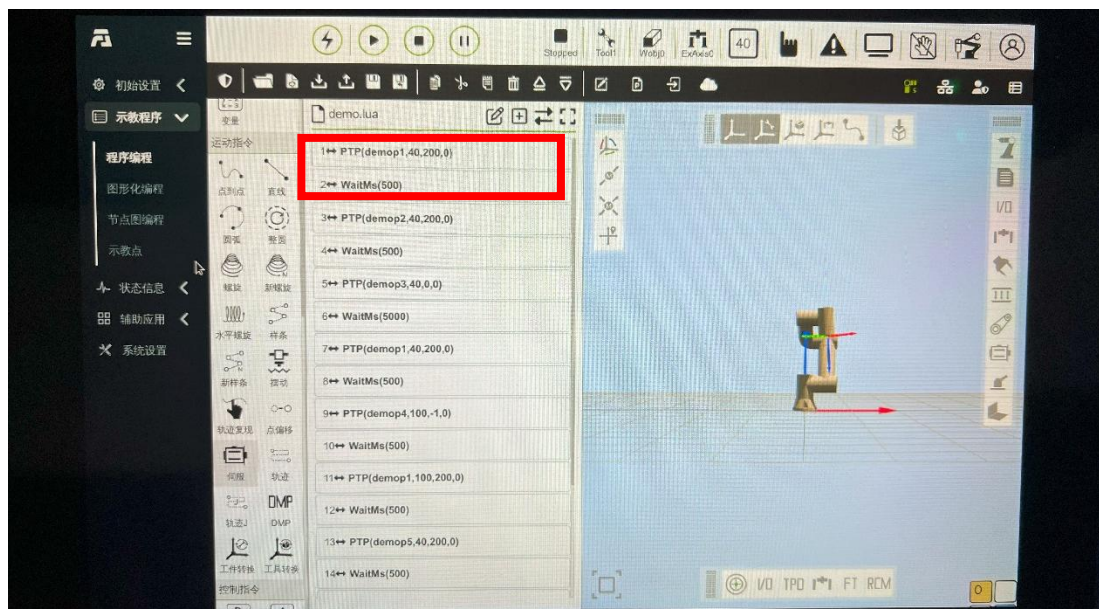


Figure 4: Operational Flow Step One Code

2. Positioning and grabbing the bottle: Program the manipulator to move to the lower left corner of the table. Once the bottle is located, instruct the manipulator

to grasp it securely.

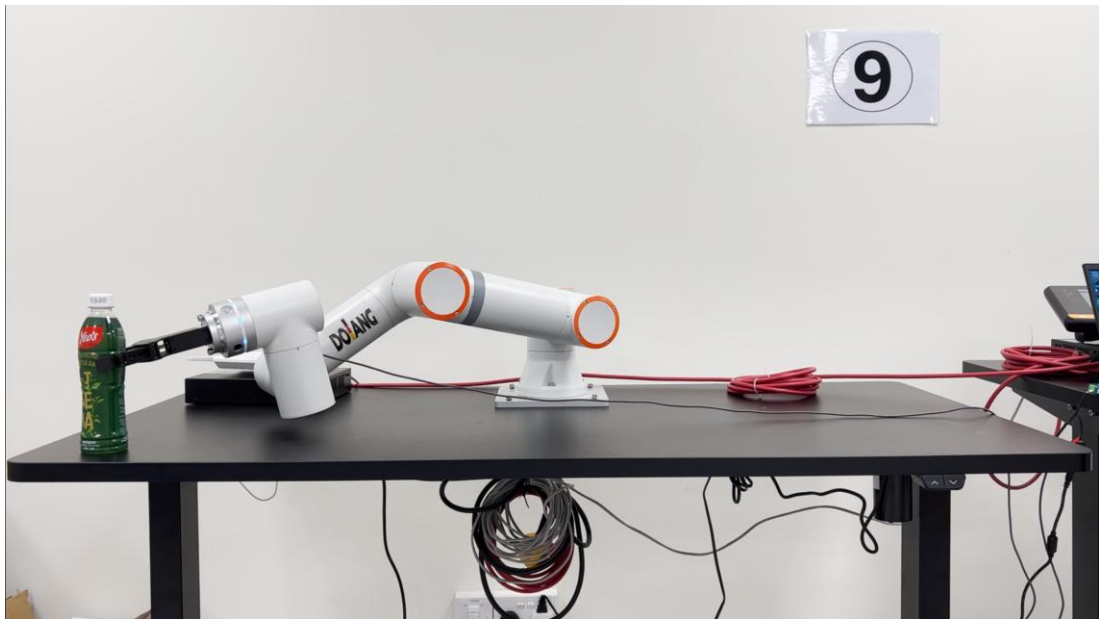


Figure 5: Operational Flow Step Two

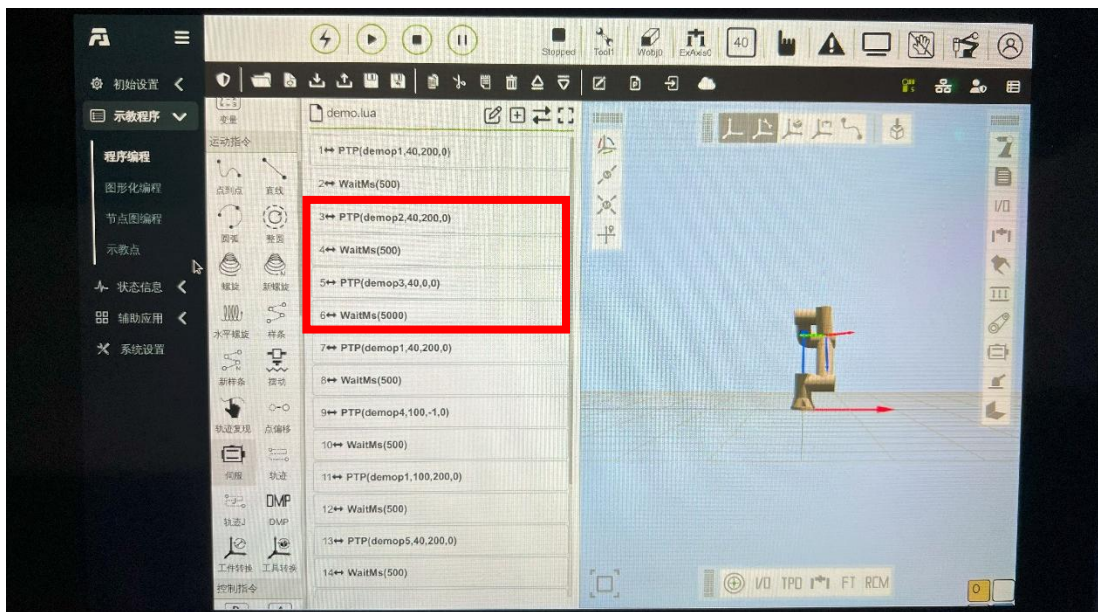


Figure 6: Operational Flow Step Two Code

3. Returning to Origin: Direct the manipulator to return to its initial position. Then, rotate the bottle 90 degrees to simulate shaking.

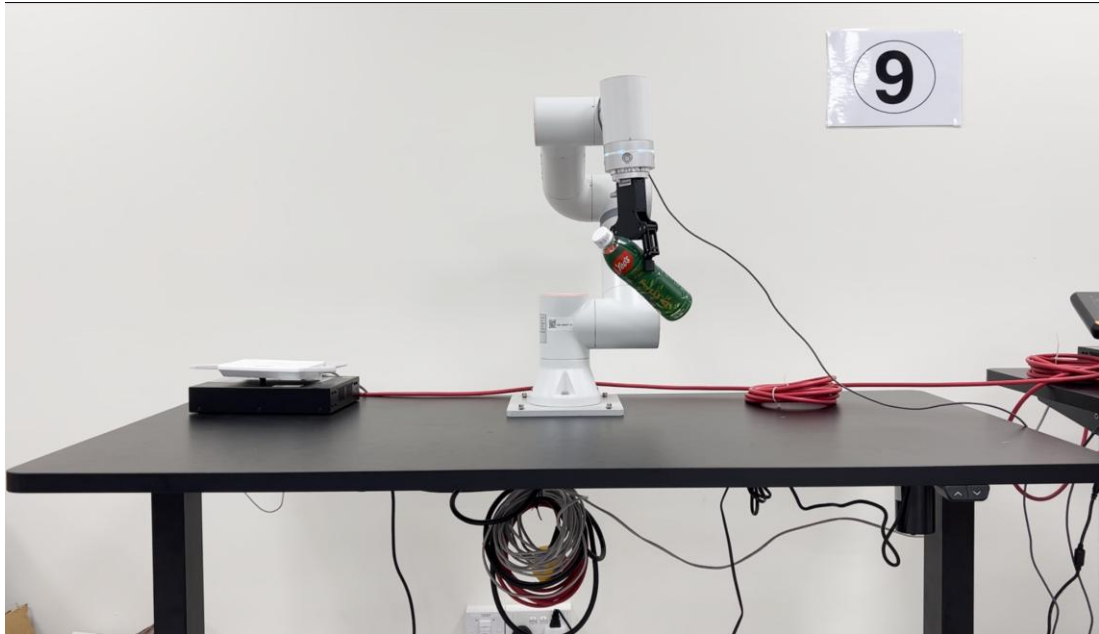


Figure 7: Operational Flow Step Three

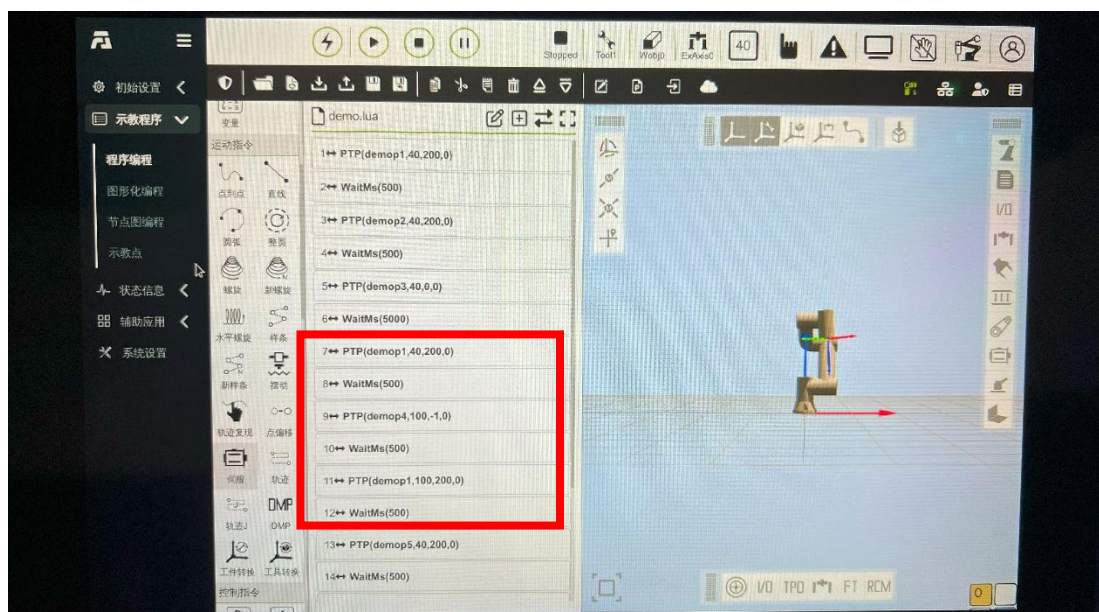


Figure 8: Operational Flow Step Three Code

4. Repositioning and Release: After shaking, program the manipulator to move to the lower right corner of the table and carefully release the bottle.

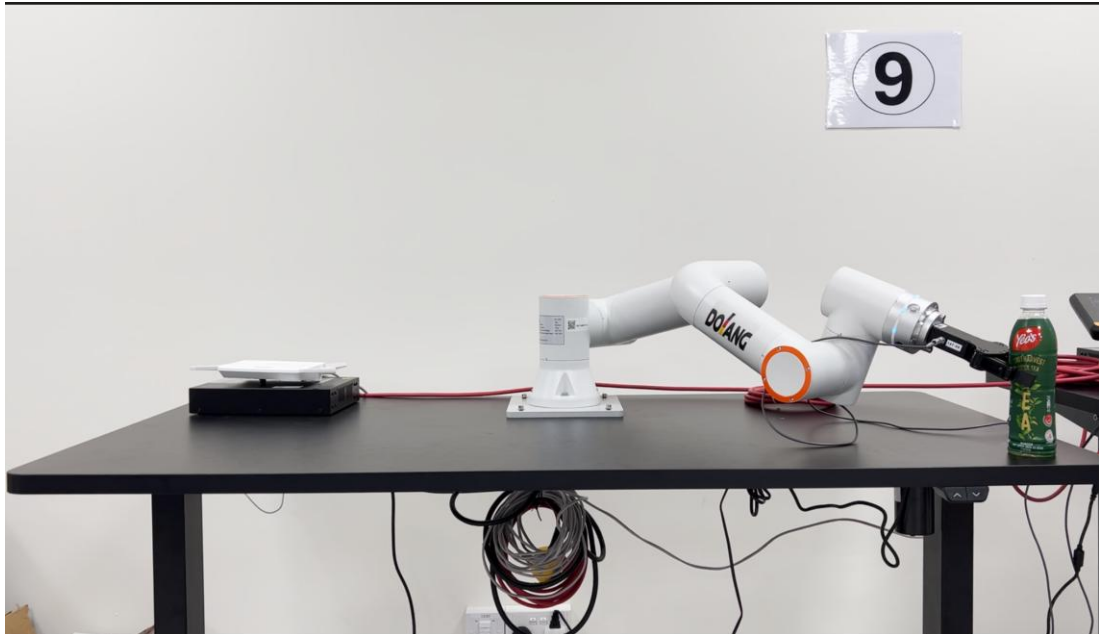


Figure 9: Operational Flow Step Four

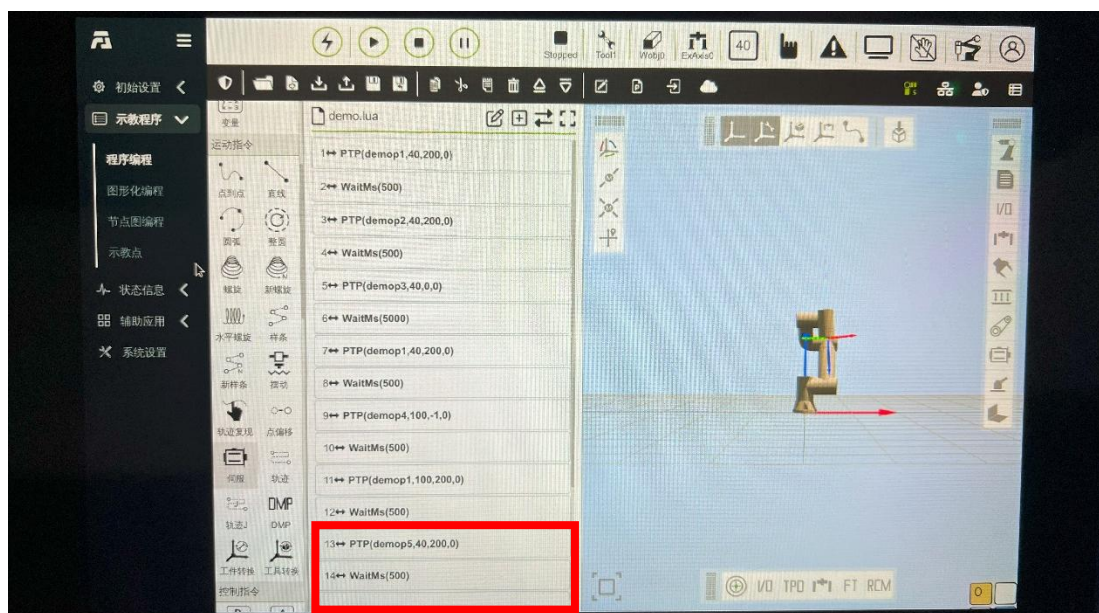


Figure 10: Operational Flow Step Four Code

5. Completion: Finally, command the manipulator to return to its original position.

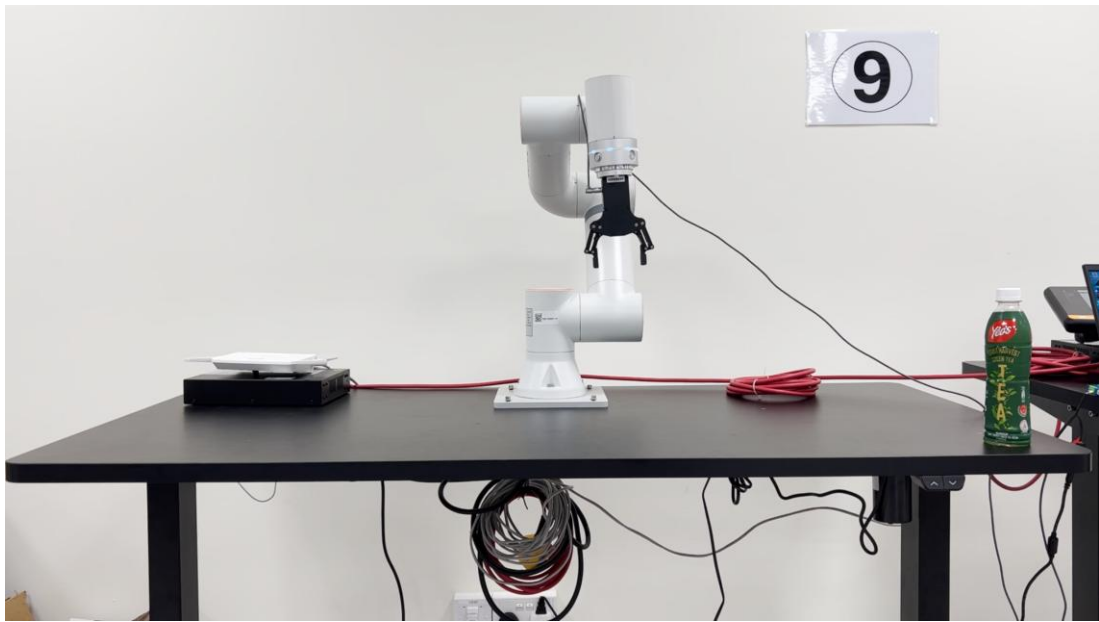


Figure 11: Operational Flow Step Five

User Configurable Settings

The programming of the cobot can be conducted via a programming language called Lua. In the meantime, to facilitate programming, a GUI interface is also provided to enable codeless programming, as shown in the following pictures. The GUI interface is shown in Figure 12, while its Lua version is shown in Figure 13.

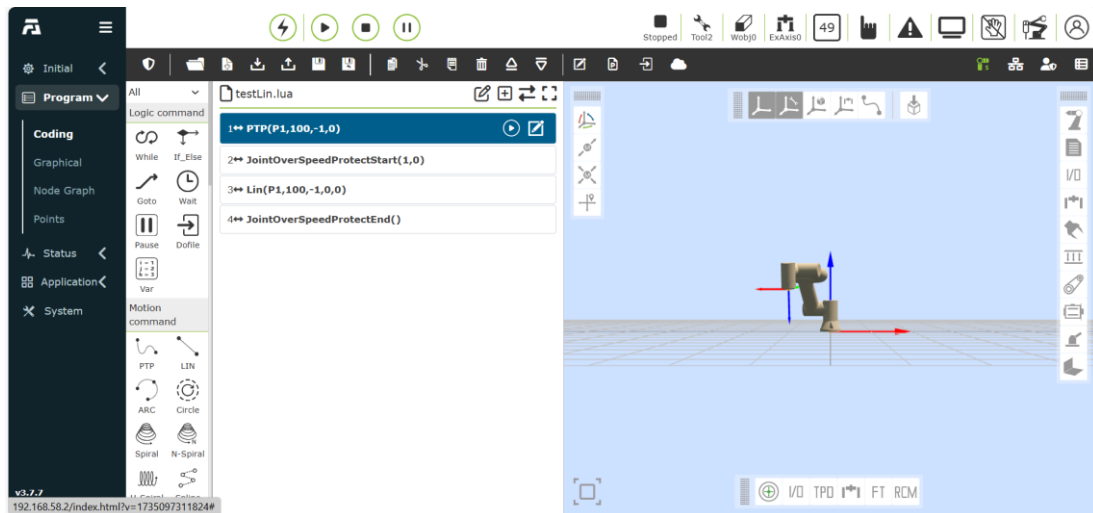


Figure 12: Programming Interface – GUI

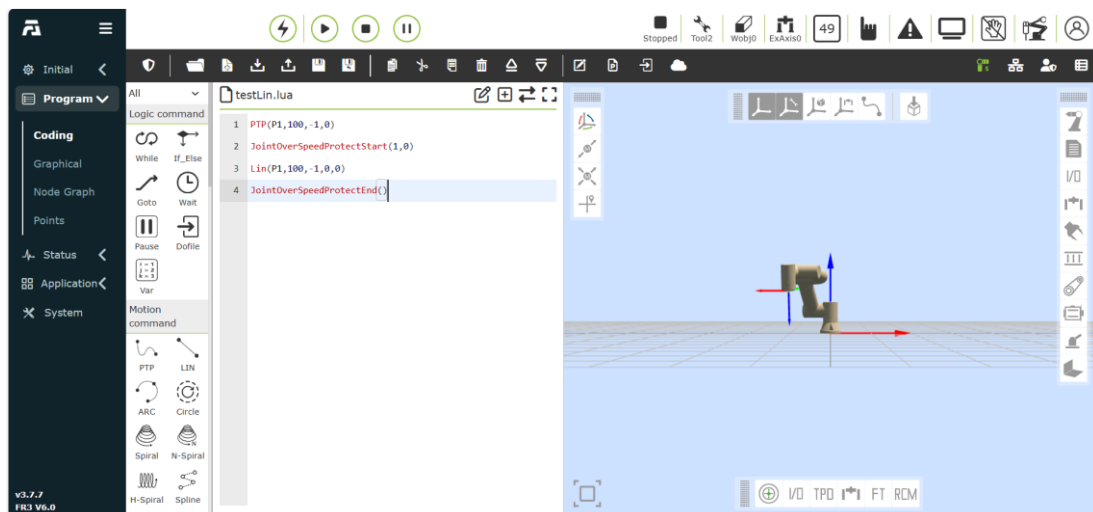


Figure 13: Programming Interface – Lua

To modify the program tree, user can use the tool bar on the interface.

			
Expand/Zoom	Add teaching points	Rename	Open
			
New build	Import	Export	Save
			
Save as	Copy	Paste	To cut
			
Delete	Move up	Move down	Toggle edit mode
			
Programming add/edit	Robot Model	NewDofile subroutine	Modbus TCP Settings
			
Current program backup	Local teaching point		

Program command

The left side is mainly for adding program commands. Click the icon above each keyword to enter the detailed interface of adding program commands on the right. There are two main operations for adding program commands to files:

1. Open the relevant command and click the Apply button to add the command to the program.
2. Click the “Add” button first, at this time the command is not saved in the program file, and you need to click “Apply” again to save the command in the file.

The second method often occurs when multiple commands of the same type are issued. We add an add button and display the content of the added command to this type of command. Click the Add button to add a command, and the added command displays all the added commands, click “Apply” to save the added command to the opened file on the right.

Logic Command Interface

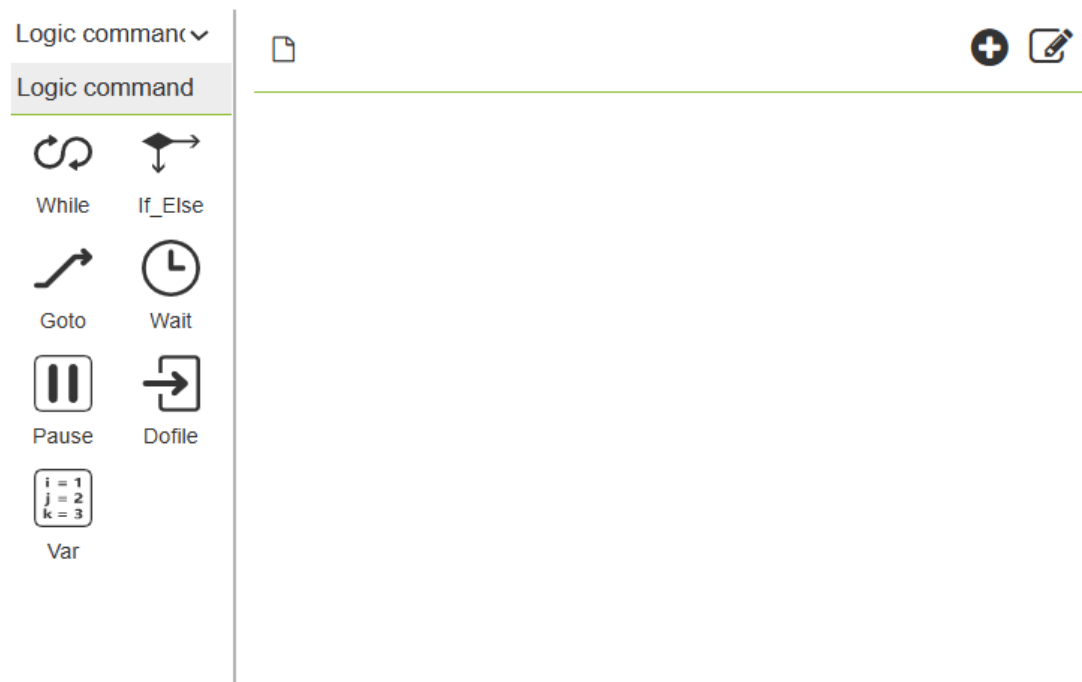


Figure 14: Logic Command Interface

While command

Select the loop scenario of the While command, the scenario is as follows:

- Always loop
- Limited number of cycles: Enter the number of loops and variable name
- Loop while expression is true: Click the input box to pop up the expression editor and select the corresponding expression according to the usage scenario

While	While
Instruction Type ☐ Always loop ☐ Limited number of cycles ☐ Loop while expression is true Add Program preview Delete Clear Apply	Instruction Type ☒ Always loop ☐ Limited number of cycles ☐ Loop while expression is true Add Program preview while (1) do end; Delete Clear Apply
While command interface	While command - Always loop

While Instruction Type ☐ Always loop ☒ Limited number of cycles Frequency 10 Variable loop1 ☐ Loop while expression is true Add Program preview <pre>loop1 = 0 while (loop1 < 10) do loop1 = loop1 + 1 end;</pre> Delete Clear Apply	While Instruction Type ☐ Always loop ☐ Limited number of cycles ☒ Loop while expression is true Click Enter Input GetDI GetAI AND OR = = ! = < > < = > = 7 8 9 4 5 6 1 2 3 0 . Close Clear Delete Confirm Add Program preview Delete Clear Apply
While command - Limited number of cycles	While command - Expression Editor
While Instruction Type ☐ Always loop ☐ Limited number of cycles ☒ Loop while expression is true GetDI(0,0) Add Program preview <pre>while (GetDI(0,0)) do end;</pre> Delete Clear Apply	
While command - Loop while expression is true	

For ease of operation, you can enter any do content and edit other instructions in the program to insert them instead.

If ... else command

Click the “if. . . else” button to enter the if. . . else command editing interface.

This command contains the following buttons:

- Add else if: When there is no “else” expression, click this button to add an “else if” expression
- Delete else if: When an “else if” expression exists, click this button to delete the “else if” expression
- Add else: Click this button to add an “else” expression
- Delete else: Click this button to delete the “else” expression

if-else

Commands Edit

Add else if

Delete else if

Add else

Delete else

if judgment expression

Click Enter

end

Add

Program preview

Delete

Clear

Apply

Figure 15: if...else Command Interface

After clicking the corresponding button to add, click the input box to pop up the expression editor and select the corresponding expression according to the usage scenario. After adding, click “Add” and “Apply”.

Goto command

Click the “Goto” button to enter the Goto command editing interface.

The Goto instruction is a jump instruction, enter the statement in the input box on the right, and click “Add” and “Apply” after editing.

Goto

Commands Edit

Label

Add

Program preview

Delete

Clear

Apply

Figure 16: Goto Command Interface

Wait command

Click the “Wait” icon to enter the Wait command editing interface.

This instruction is a delay instruction, which is divided into three parts: “WaitMs”, “WaitDI” and “WaitAI”.

Wait

Instruction Type

☒ Time to wait

waiting time (ms)

0-10000

☐ Single DI waiting

☐ Multi DI wait

☐ AI Waits

Add

Program preview

Delete

Clear

Apply

Figure 17: WaitTime Command Interface

“WaitTime” command delay waiting time unit is milliseconds, input the number of milliseconds to wait, click “Add”, “Apply”.

Wait

Instruction Type

☐ Time to wait

☒ Single DI waiting

☐ Multi DI wait

Port

Ctrl-DI0

State

False

Maximum time (ms)

0-10000

Wait for timeout processing

Stop error reporting

Add

Program preview

Delete

Clear

Apply

Figure 18: WaitDI Command Interface

“WaitDI” command, that is, single DI waiting, select the IO port number to be waited for, wait state, wait maximum time and wait timeout processing method, and click “Add” and “Apply”.

Wait

Instruction Type

☐ Time to wait

☐ Single DI waiting

☒ Multi DI wait

Conditional selection

And

☐ DI0

False

☐ CI0

False

☐ DI1

False

☐ CI1

False

☐ DI2

False

☐ CI2

False

☐ DI3

False

☐ CI3

False

Add

Program preview

Delete

Clear

Apply

Figure 19: WaitMultiDI Command Interface

“WaitMultiDI” command, that is, multi-DI waiting, first select the multi-DI establishment conditions, then check the DI port and status that need to wait, and finally set the maximum waiting time and waiting timeout processing method, click “Add” and “Apply”.

Wait

☐ Single DI waiting
 ☐ Multi DI wait
 ☒ AI Waits

Port

condition

numerical value (ms)

Wait for timeout processing

Maximum time (ms)

Add

Program preview

Delete

Clear

Apply

Figure 20: WaitAI Command Interface

“WaitAI” command, select the analog quantity to be waited for, the value, the maximum waiting time and the waiting timeout processing method, and click “Add” and “Apply”.

Pause command

Click the “Pause” icon to enter the Pause command editing interface.

This instruction is a pause instruction. Insert this instruction into the program. When the program executes this instruction, the robot will be in a pause state. If you want to continue running, click the “Pause/Resume” button in the control area.

Pasue

Commands Edit

Pause function

No function

Add

Program preview

DeleteClearApply

Figure 21: Pause Command Interface

Dofile command

Click the “Dofile” icon to enter the Dofile command editing interface.

The Dofile command calls the internal program of the controller. When using the Dofile command, the called subroutine needs to be saved, and the main program does not need to be saved again if it has not changed. The Dofile command supports two-level calls, and two parameter settings need to be paid attention to. One is the level of the call, and the other is the ID number of the call. In principle, the same ID number cannot appear in the same program.

Dofile

Commands Edit

Dofile lua file
aa.lua

File content preview
while (1) do
end
while (SPLCGetDI(8,1,123)) do
end
while (SPLCGetAI(1,0,10,1000)) do
end

Which layer is called

ID No
1-10000

ID cannot be repeated in the same program

Add

Program preview

Delete

Clear

Apply

Figure 22: Dofile Comand Interface

Var command

Click the “Var” icon to enter the Var command editing interface.

This command is a variable system command, which is divided into two parts: Lua variable definition, variable query and Sys variable renaming, getting value, and setting value. Lua variable definition can declare a variable and assign an initial value, and cooperate with while, if-else and other commands Use the Lua variable query command to query the value of the input variable name in real time and display it in the status bar. The number of Sys variables is fixed, and you can rename them, get variable values, and set variable values. The values stored in this variable will not be cleared when the system is turned off.

Variable

Instruction Type

- ☐ Lua variable
- ☐ Lua user variable query
- ☐ User variable settings
- ☐ User variable acquisition
- ☐ System variable acquisition
- ☐ System variable settings
- ☐ Input register variable read
- ☐ Output register variable read
- ☐ Output register variable write

Program preview

Delete

Clear

Apply

Figure 23: Var Command Interface

Motion command interface

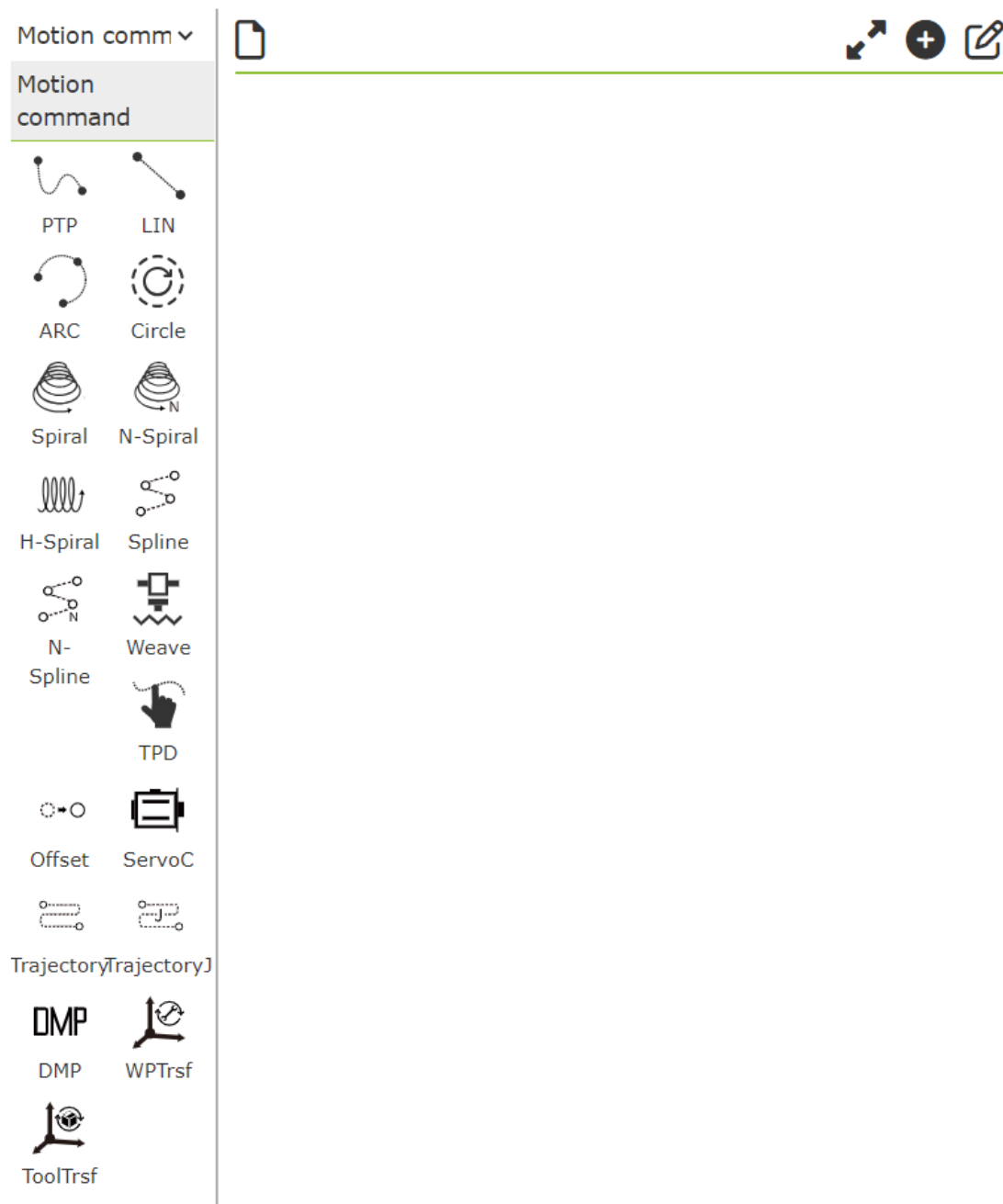


Figure 24: Motion Command Interface

PTP command

Click the “PTP” icon to enter the PTP command editing interface.

You can choose the point to be reached, and the smooth transition time setting can realize that the movement from this point to the next point is continuous. Whether to set the offset, you can choose to offset based on the base coordinate system and based on the tool coordinates, and pop up x, y, z, rx, ry, rz offset settings, PTP specific path is the optimal path automatically planned by the motion controller, click “Add” and “Apply” to save this command.

PTP

Commands Edit

Point Name
44

Commissioning speed (%)
100

At this point
☒ Stop
☐ Smooth(ms)

Offset or not
no

Motion protection
☒ None
☐ Time Optimization Mode

Program preview

Tool

coordinate system:
toolcoord0

Workpiece coordinate system: 0

J1: 54.035

J2: -85.520

J3: -0.163

J4: -95.569

J5: -99.629

Add

Delete

Clear

Apply

Figure 25: PTP Command Interface

Multiple motion command is provided in the system. For detail information of other motion commands, please refer to the cobot user manual.

Experimental Tasks

A. Cobot arm movement with electrical gripper utilization via software

To perform precise pick-and-place or manipulation tasks using an electrically powered gripper. The cobot follows a planned motion path such as linear or joint movements to reach the object. The electrical gripper is controlled directly through the robot's software interface, allowing users to set parameters like grip force, speed, and opening width. Through software integration, the cobot can execute actions such as grip, release, or detect object presence with high precision and repeatability. This setup is ideal for handling delicate components in electronics, packaging, and small-parts assembly.

Necessary Skills

Lua Programming

The GUI-based programming interface provided by the cobot manufacturer, Aubo, utilizes Lua as its scripting backend. Therefore, students are strongly encouraged to develop proficiency in Lua in order to gain a deeper understanding of the underlying programming logic and to effectively write custom scripts for completing assigned tasks.

Overview of Lua

Lua is a lightweight, high-level, dynamically typed programming language designed for embedded systems and configurable applications. Originally developed in 1993 at the Pontifical Catholic University of Rio de Janeiro (PUC-Rio), Lua has gained significant popularity in domains such as game development, IoT, automation, and robotics due to its small footprint, portability, and embeddability in host applications written in C or C++.

Why does Aubo use Lua for cobot programming?

- **Lightweight:** Minimal memory usage—ideal for microcontrollers and embedded systems.
- **Fast:** Lua has a just-in-time (JIT) compiler for high performance on constrained hardware.
- **Simple Syntax:** Easy to learn and read; especially beneficial for rapid prototyping.
- **Interoperable:** Integrates easily with C/C++ robotics frameworks.
- **Flexible:** Dynamically typed, functional and procedural programming supported.

Language Fundamentals

1. Hello World



```
1 print("Hello, Robot!")
```

Figure 26: Lua Codes 1

2. Variables and Types

Lua has eight basic types: *nil*, *boolean*, *number*, *string*, *function*, *userdata*, *thread*, and *table*.

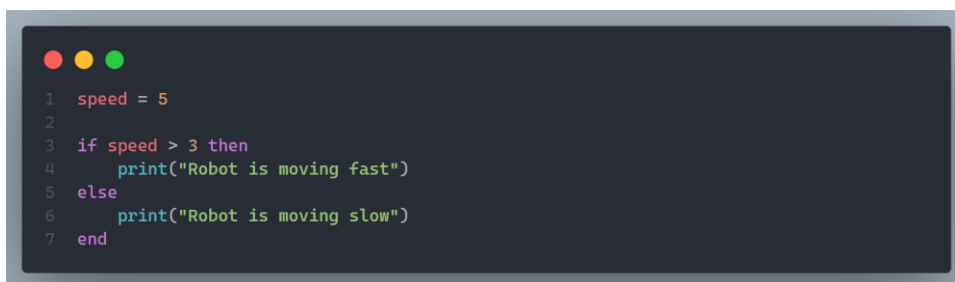
Note that all variables are global by default unless explicitly declared as local.



```
1 x = 10      -- number
2 y = "forward" -- string
3 flag = true  -- boolean
```

Figure 27: Lua Codes 2

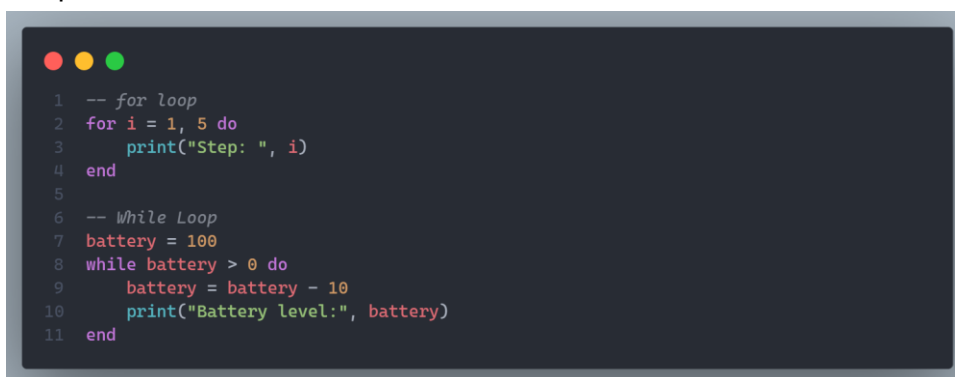
3. If-else Structures



```
1 speed = 5
2
3 if speed > 3 then
4     print("Robot is moving fast")
5 else
6     print("Robot is moving slow")
7 end
```

Figure 28: Lua Codes 3

4. Loops



```
1 -- for loop
2 for i = 1, 5 do
3     print("Step: ", i)
4 end
5
6 -- While Loop
7 battery = 100
8 while battery > 0 do
9     battery = battery - 10
10    print("Battery level:", battery)
11 end
```

Figure 29: Lua Codes 4

Data Structures

Tables are the only data structure in Lua and can represent arrays, dictionaries (maps), and objects. Tables can be defined in both array-style and key-value style:

```
1 positions = {0, 10, 20, 30}
2 print(positions[2]) -- Output: 10
```

Figure 30: Lua Codes 5

```
1 robot = {name = "Alpha", speed = 2.5}
2 print(robot["name"]) -- Output: Alpha
```

Figure 31: Lua Codes 6

Functions

Functions are first-class citizens and can be defined inline or stored in variables.

```
1 Robot = {}
2 local max_speed = 3.0
3
4 -- Speed limiter (closure)
5 function Robot.limitSpeed(speed)
6     if speed > max_speed then return max_speed
7     elseif speed < -max_speed then return -max_speed
8     else return speed end
9 end
10
11 -- Core actuator
12 function Robot.setWheels(l, r)
13     -- Apply limiter
14     local L = Robot.limitSpeed(l)
15     local R = Robot.limitSpeed(r)
16     -- Replace with real actuation API
17     print(("Wheels -> L: %.2f, R: %.2f"):format(L, R))
18 end
19
20 -- Sensor-driven callback behavior
21 function Robot.avoidObstacle(sensor, onHit, onClear)
22     if sensor() < 0.2 then onHit() else onClear() end
23 end
24
25 -- Simple behaviors
26 function Robot.stop() Robot.setWheels(0, 0) end
27 function Robot.forward() Robot.setWheels(2, 2) end
28 function Robot.turnLeft() Robot.setWheels(-1, 1) end
29
30 -- Mock sensor
31 function readDist() return math.random() end
32
33 -- Autonomous control in one step
34 function Robot.step()
35     Robot.avoidObstacle(readDist,
36         function() Robot.turnLeft() end,
37         function() Robot.forward() end
38     )
39 end
40
41 -- Demo loop
42 for i = 1, 5 do
43     Robot.step()
44 end
```

Figure 32: Lua Codes 7

Example Scripts

Inter-process or inter-device communication is pervasively used when programming the working logic of the cobot. For instance, the cobot's movement could be triggered by a signal sent from a controller located in another device. Following example scripts provide a simplified UDP server and UDP Client that can be used for communication.

UDP Server

```
1  local socket = require("socket")
2
3  -- Create a UDP socket and bind it to localhost on port 12345
4  local udp = assert(socket.udp())
5  udp:setsockname("127.0.0.1", 12345)
6  udp:settimeout(0) -- Non-blocking mode
7
8  print("Listening for incoming messages on port 12345...")
9
10 while true do
11     local data, ip, port = udp:receivefrom()
12     if data then
13         print(string.format("Received from %s:%d: %s", ip, port, data))
14         -- You can add more logic here to act on the message
15     end
16     socket.sleep(0.1) -- Prevents 100% CPU usage
17 end
```

Figure 33: Lua Codes 8

UDP Client

```
1  local socket = require("socket")
2
3  -- Create a UDP socket
4  local udp = assert(socket.udp())
5
6  -- Set the target IP and port (must match the listener)
7  local target_ip = "127.0.0.1"
8  local target_port = 12345
9
10 -- Message to send
11 local message = "Hello from the sender!"
12
13 -- Send the message
14 assert(udp:sendto(message, target_ip, target_port))
15 print(string.format("Sent to %s:%d: %s", target_ip, target_port, message))
16
17 udp:close()
```

Figure 34: Lua Codes 9

B. Cobot arm movement with electrical gripper utilization via communication protocol

The electrical gripper is operated using a communication protocol such as Modbus TCP, RS-485, or Ethernet/IP. The cobot executes programmed motion paths to reach target positions, and sends commands to the gripper through the established protocol interface. These commands control actions like opening/closing the gripper, adjusting grip force, and receiving feedback (e.g., object detected or error status). This integration allows for synchronized operation between the cobot and the gripper, enabling high precision, real-time control, and seamless automation in applications such as electronics handling, packaging, or assembly.

Necessary Skills

Lua Programming

Please refer to section Lua Programming.

Python Programming

Overview of Python

Python is a high-level, interpreted programming language known for its emphasis on readability, simplicity, and developer productivity. It was created by Guido van Rossum and first released in 1991. The language design prioritizes code that is easy to understand and maintain.

Fundamentals

Python is more sophisticated comparing to Lua, hence, it's fundamentals is not introduced here due to the limitation of space. Please refer to [this](#) video for more details.

Examples

Inter-process or inter-device communication is pervasively used when programming the working logic of the cobot. For instance, the cobot's movement could be triggered by a signal sent from a controller located in another device. Following example scripts provide a simplified TCP server and TCP Client that can be used for communication.

TCP Server

```
1 # tcp_server.py
2 import socket
3
4 HOST = '127.0.0.1' # Localhost
5 PORT = 65432      # Arbitrary non-privileged port
6
7 def start_server():
8     with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server_sock:
9         server_sock.bind((HOST, PORT))
10        server_sock.listen()
11        print(f"Server listening on {HOST}:{PORT}...")
12
13        conn, addr = server_sock.accept()
14        with conn:
15            print(f"Connected by {addr}")
16            while True:
17                data = conn.recv(1024)
18                if not data:
19                    print("Client disconnected.")
20                    break
21                print(f"Received: {data.decode()}")
22                conn.sendall(data) # Echo back
23
24 if __name__ == "__main__":
25     start_server()
```

Figure 35: Python Example Code 1

TCP Client

```
1
2 # tcp_client.py
3 import socket
4
5 HOST = '127.0.0.1' # Must match the server
6 PORT = 65432
7
8 def start_client():
9     with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as client_sock:
10        client_sock.connect((HOST, PORT))
11        print("Connected to the server.")
12        try:
13            while True:
14                message = input("Enter message (or 'exit'): ")
15                if message.lower() == 'exit':
16                    break
17                client_sock.sendall(message.encode())
18                data = client_sock.recv(1024)
19                print(f"Echo from server: {data.decode()}")
20        except KeyboardInterrupt:
21            print("\nDisconnected from server.")
22
23 if __name__ == "__main__":
24     start_client()
25
```

Figure 36: Python Example Code 2

Communication Protocols

In robotic systems engineering, communication protocols constitute a critical foundation enabling effective interoperability among heterogeneous system components, such as robots, controllers, sensors, and actuators. Advanced knowledge

of these protocols is essential for developing sophisticated robotic systems capable of autonomous and semi-autonomous operation within complex, dynamic environments. Furthermore, a comprehensive grasp of these communication standards greatly enhances an engineer's capacity for system diagnostics, optimization of data throughput, latency minimization, and implementation of robust cybersecurity mechanisms.

Structure of Internet-based Communication

Robotic communication protocols frequently adopt the layered architecture articulated by the TCP/IP reference model, typically encompassing the following layers:

- **Application Layer:** Operating at the highest abstraction, this layer directly interfaces with software applications. Protocols such as HTTP, MQTT, FTP, and others facilitate structured data exchange, system remote control, and integration with external service-oriented architectures.
- **Transport Layer:** Responsible for the end-to-end management of data transmission, this layer ensures data integrity, reliability, and sequencing through protocols such as Transmission Control Protocol (TCP) and User Datagram Protocol (UDP). It addresses the complexities associated with data segmentation, error control, flow control, and multiplexing.
- **Network Layer:** This intermediate layer provides logical addressing and routing functionalities, determining optimal paths for packet delivery across diverse networks. The primary protocol operating at this layer is the Internet Protocol (IP), which supports efficient network-level data communication.
- **Link Layer:** At the foundational level, the link layer defines rules for data transmission within local network segments. It encompasses protocols such as Ethernet and Wi-Fi, managing the physical addressing and media access control necessary for communication within local or wireless networks.

In robotics, lower-layer protocols like TCP and UDP are foundational for real-time system performance, while protocols in applications layer such as HTTP and similar protocols are more common in integration with user interfaces, cloud services, or diagnostics.

Different Communication Protocols

Transmission Control Protocol

TCP is one of the foundational communication protocols in the Internet protocol suite. It provides reliable, ordered, and error-checked delivery of a stream of bytes between applications running on hosts communicating via an IP network. In the context of robotic programming, TCP is frequently employed for scenarios that require robust and consistent communication, particularly when data integrity and delivery order are critical.

It has some key features:

- **Connection-Oriented Communication**
TCP establishes a connection between two endpoints using a three-way handshake process before any data is transmitted. This ensures that both parties are ready for communication and agree on parameters such as sequence numbers and window size.
- **Reliability**
TCP guarantees the delivery of data to the destination. It uses sequence numbers and acknowledgments (ACKs) to confirm receipt of packets. If an acknowledgment is not received within a specific timeframe, the data is retransmitted.
- **In-Order Delivery**
Data sent over a TCP connection arrives at the receiving end in the same order in which it was sent. This is crucial for many robotic applications where the semantic integrity of command sequences must be preserved.
- **Flow Control**
TCP employs a flow control mechanism using a sliding window protocol. This prevents a fast sender from overwhelming a slow receiver, ensuring smooth data transfer.
- **Congestion Control**
TCP dynamically adjusts the rate of data transmission based on network congestion conditions. Algorithms like Slow Start, Congestion Avoidance, and Fast Retransmit are integral to this functionality.

TCP is often used in robotic systems for:

- **Command and Control Interfaces:** For example, sending joint angle commands or velocity profiles from a control station to a robotic arm, where each command must be received accurately and in sequence.
- **Sensor Data Transmission:** Where lossless transmission of critical sensor data (e.g., from LiDARs or high-fidelity IMUs) is required for mapping, localization, or perception.
- **Remote Operation over LAN/WAN:** Teleoperation systems frequently rely on TCP when operating in environments where intermittent packet loss would be detrimental to robot stability or performance.

Example Code:

```

1  # tcp_server.py
2  import socket
3
4  HOST = '127.0.0.1' # Localhost
5  PORT = 65432      # Arbitrary non-privileged port
6
7  def start_server():
8      with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server_sock:
9          server_sock.bind((HOST, PORT))
10         server_sock.listen()
11         print(f"Server listening on {HOST}:{PORT}...")
12
13         conn, addr = server_sock.accept()
14         with conn:
15             print(f"Connected by {addr}")
16             while True:
17                 data = conn.recv(1024)
18                 if not data:
19                     print("Client disconnected.")
20                     break
21                 print(f"Received: {data.decode()}")
22                 conn.sendall(data) # Echo back
23
24 if __name__ == "__main__":
25     start_server()

```

Figure 37: TCP Server

```

1
2  # tcp_client.py
3  import socket
4
5  HOST = '127.0.0.1' # Must match the server
6  PORT = 65432
7
8  def start_client():
9      with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as client_sock:
10         client_sock.connect((HOST, PORT))
11         print("Connected to the server.")
12         try:
13             while True:
14                 message = input("Enter message (or 'exit'): ")
15                 if message.lower() == 'exit':
16                     break
17                 client_sock.sendall(message.encode())
18                 data = client_sock.recv(1024)
19                 print(f"Echo from server: {data.decode()}")
20         except KeyboardInterrupt:
21             print("\nDisconnected from server.")
22
23 if __name__ == "__main__":
24     start_client()
25

```

Figure 38: TCP Client

User Datagram Protocol

UDP is a core member of the Internet protocol suite and serves as a connectionless, lightweight transport protocol. Unlike TCP, UDP does not guarantee reliable or ordered delivery of packets. It is designed for applications that require fast, low-latency communication and can tolerate occasional data loss or reordering.

In robotic programming, UDP is commonly used in scenarios where speed and responsiveness are more critical than guaranteed delivery—particularly in real-time systems and high-frequency sensor streams.

It has some key features:

- **Connectionless Communication**
UDP does not establish a persistent connection between sender and receiver. Instead, it sends datagrams independently, without handshakes or session management, which significantly reduces communication overhead.
- **No Reliability Guarantee**
Packets sent via UDP may be lost, duplicated, or delivered out of order. The protocol does not implement acknowledgments or retransmissions. If reliability is needed, it must be implemented at the application level.
- **Low Latency and Minimal Overhead**
By forgoing connection management and reliability features, UDP provides lower latency and higher throughput compared to TCP, making it suitable for time-sensitive robotic applications.
- **Message-Based Transmission**
Unlike TCP, which presents data as a continuous stream, UDP preserves message boundaries. Each datagram is self-contained and delivered as a discrete unit.
- **No Congestion or Flow Control**
UDP does not provide built-in congestion or flow control. It sends data as fast as the application allows, which can result in packet loss if the network or receiver cannot keep up.

UDP is widely adopted in robotics for:

- **Real-Time Control and Feedback:** For example, streaming motor feedback, sensor measurements, or robot pose estimates at high rates where latency must be minimized.
- **Broadcast and Multicast Communication:** UDP supports broadcast and multicast, allowing data (e.g., localization updates or discovery signals) to be sent efficiently to multiple recipients.
- **Simulation and Visualization:** Tools such as simulators or visualization platforms may receive pose or state updates via UDP to maintain real-time performance.
- **Time-Sensitive Networking:** In some robotic applications, UDP is used as the transport layer in conjunction with time synchronization protocols and Quality of Service (QoS) mechanisms.

Example Code:

```

1 # udp_server.py
2 import socket
3
4 HOST = '127.0.0.1' # Localhost
5 PORT = 65433      # Use a different port from TCP example
6
7 def start_udp_server():
8     with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as server_sock:
9         server_sock.bind((HOST, PORT))
10        print(f"UDP server listening on {HOST}:{PORT}...")
11
12        while True:
13            data, addr = server_sock.recvfrom(1024)
14            print(f"Received from {addr}: {data.decode()}")
15            server_sock.sendto(data, addr) # Echo back
16
17 if __name__ == "__main__":
18     start_udp_server()

```

Figure 39: UDP Server

```

1 # udp_client.py
2 import socket
3
4 HOST = '127.0.0.1'
5 PORT = 65433
6
7 def start_udp_client():
8     with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as client_sock:
9         print("UDP client started. Type 'exit' to quit.")
10        while True:
11            message = input("Enter message: ")
12            if message.lower() == 'exit':
13                break
14            client_sock.sendto(message.encode(), (HOST, PORT))
15            data, _ = client_sock.recvfrom(1024)
16            print(f"Echo from server: {data.decode()}")
17
18 if __name__ == "__main__":
19     start_udp_client()
20

```

Figure 40: UDP Client

Hypertext Transfer Protocol

HTTP is an application-layer protocol used for distributed, collaborative, and hypermedia information systems. Originally designed for web communication, HTTP has also found utility in robotic programming—particularly for interfacing with web services, cloud-based APIs, and RESTful architectures.

Unlike TCP or UDP, which operate at the transport layer and handle low-level data transmission, HTTP builds on top of transport protocols (typically TCP) and defines the semantics of client-server communication, including methods (e.g., GET, POST), status codes, headers, and content formats.

It has some key features:

- **Application-Layer Protocol**

HTTP defines structured, high-level message exchanges between clients and servers. It dictates *what* is being communicated (e.g., resource requests or sensor uploads), not *how* the bytes are transmitted (which is handled by the transport layer).

- **Stateless Communication**

Each HTTP request-response pair is independent, with no built-in notion of session or state across multiple transactions. Any required state must be managed explicitly by the application or through additional mechanisms (e.g., cookies or tokens).

- **Text-Based and Human-Readable**

HTTP messages are encoded in a text-based format (ASCII), which facilitates debugging and interoperability. However, this also increases overhead and can limit performance in bandwidth-constrained environments.

- **RESTful Interface Support**

HTTP is the foundation of REST (Representational State Transfer), an architectural style that models services as resources with standard operations (e.g., GET for reading, POST for creating). RESTful APIs are common in modern robotic systems, especially in cloud-robotics and fleet management.

HTTP is commonly employed in robotics for:

- **Interfacing with Web Services:** Robots may query cloud-based services for tasks such as route planning, map updates, or object recognition.
- **Remote Monitoring and Diagnostics:** HTTP is often used to expose robot status via REST APIs or web dashboards, allowing developers or operators to inspect logs, sensor data, or internal metrics.
- **Command and Configuration Interfaces:** HTTP endpoints are used to send high-level commands or modify configuration parameters, especially in service robots or autonomous vehicles.
- **Data Upload and Logging:** Telemetry data or diagnostic logs can be periodically uploaded to a remote server using HTTP POST requests.

Example Codes


```

1 # http_server.py
2 from flask import Flask, request, jsonify
3
4 app = Flask(__name__)
5
6 @app.route('/status', methods=['GET'])
7 def get_status():
8     return jsonify({"robot_status": "OK", "battery": 87})
9
10 @app.route('/command', methods=['POST'])
11 def receive_command():
12     data = request.get_json()
13     command = data.get("command")
14     print(f"Received command: {command}")
15     return jsonify({"received": command, "executed": True})
16
17 if __name__ == '__main__':
18     app.run(host='127.0.0.1', port=5000)

```

Figure 41: HTTP Server

```

1 # http_client.py
2 import requests
3
4 SERVER_URL = 'http://127.0.0.1:5000'
5
6 def query_robot_status():
7     response = requests.get(f"{SERVER_URL}/status")
8     print("Status Response:", response.json())
9
10 def send_robot_command(command):
11     response = requests.post(f"{SERVER_URL}/command", json={"command": command})
12     print("Command Response:", response.json())
13
14 if __name__ == "__main__":
15     query_robot_status()
16     send_robot_command("MOVE_FORWARD")
17     send_robot_command("TURN_LEFT")

```

Figure 42: HTTP Client

RS-485 Communication Protocol

Introduction

RS-485 (Recommended Standard 485) is an electrical standard defining the physical layer of serial communication. It's not a protocol in the sense of Modbus or CANopen—rather, it's the physical and electrical characteristics for differential signaling over a pair of wires.

Key Characteristics:

- **Differential signaling:** RS-485 uses two wires (A and B) to transmit data differentially. This increases noise immunity and allows longer distances (~1.2 km) and higher speeds (~10 Mbps at shorter distances).
- **Multi-point capability:** Unlike RS-232, RS-485 supports multiple transceivers on the same bus (typically 32 devices).

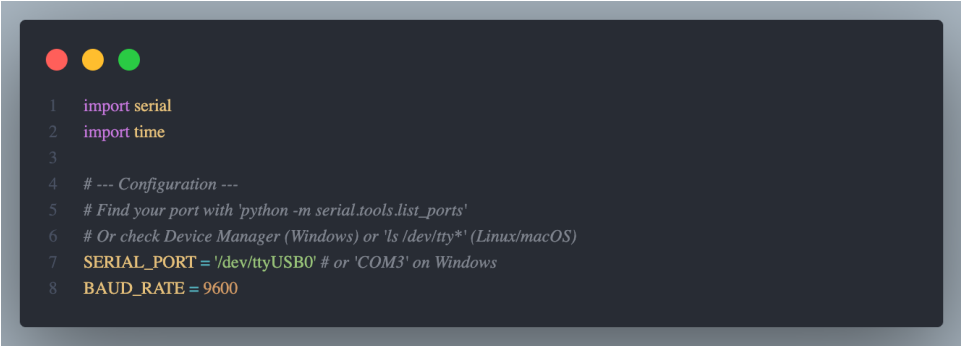
- **Half-duplex by default:** Devices take turns transmitting and receiving on the same pair of wires. Full-duplex is possible with more wires, but not common.
- **No framing/protocol:** RS-485 itself only handles the physical layer. Higher-level protocols (e.g., Modbus RTU) must be layered on top for structured communication.

Using RS-485 with Python

You can interact with RS-485 devices using a USB-to-RS485 adapter or an RS-485 transceiver connected to a UART interface (e.g., via Raspberry Pi or a microcontroller).

Python's main tool for serial communication is *pyserial*, which works for both RS-232 and RS-485.

Import Packages and Setup Port



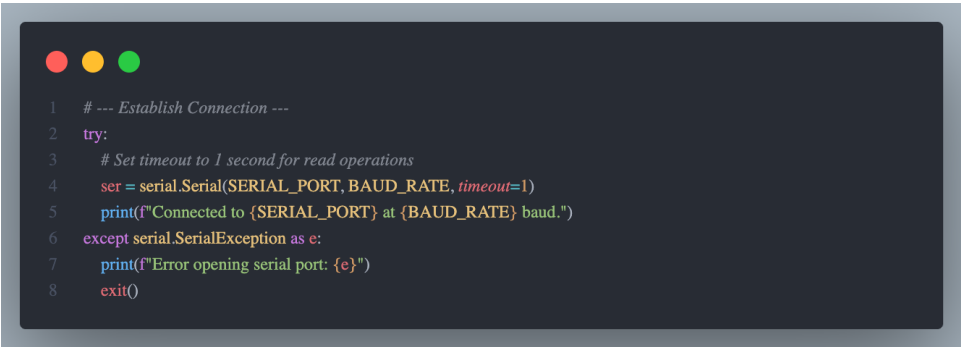
```

1 import serial
2 import time
3
4 # --- Configuration ---
5 # Find your port with 'python -m serial.tools.list_ports'
6 # Or check Device Manager (Windows) or 'ls /dev/tty*' (Linux/macOS)
7 SERIAL_PORT = '/dev/ttyUSB0' # or 'COM3' on Windows
8 BAUD_RATE = 9600

```

Figure 43: RS-485 Communication 1

Establish Communication



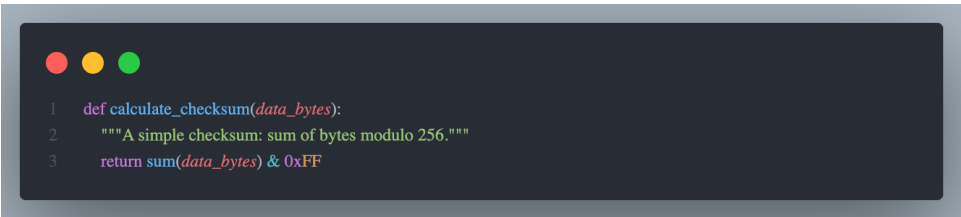
```

1 # --- Establish Connection ---
2 try:
3     # Set timeout to 1 second for read operations
4     ser = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=1)
5     print(f"Connected to {SERIAL_PORT} at {BAUD_RATE} baud.")
6 except serial.SerialException as e:
7     print(f"Error opening serial port: {e}")
8     exit()

```

Figure 44: RS-485 Communication 2

Prepare Function for Sum Checking



```

1 def calculate_checksum(data_bytes):
2     """A simple checksum: sum of bytes modulo 256."""
3     return sum(data_bytes) & 0xFF

```

Figure 45: RS-485 Communication 3

Send Custom Frame

```
1 def send_custom_frame(device_id, command, payload):
2     """Constructs and sends a custom data frame."""
3     start_byte = 0xAA
4
5     # Construct the core message
6     message = [device_id, command] + payload
7
8     # Calculate checksum on the message part
9     checksum = calculate_checksum(message)
10
11    # Final frame to be sent
12    frame = bytes([start_byte] + message + [checksum])
13
14    print(f"Sending Frame: {frame.hex(' ')}")
15    ser.write(frame)
16    time.sleep(0.1) # Give the device time to respond
```

Figure 46: RS-485 Communication 4

Receive Custom Frame

```
1 def receive_custom_frame(expected_len):
2     """Reads and parses a custom frame from the serial port."""
3     response_bytes = ser.read(expected_len)
4
5     if not response_bytes:
6         print("No response from device.")
7         return None
8
9     print(f"Received Bytes: {response_bytes.hex(' ')}")
10
11    # --- Frame Validation (customize this part) ---
12    if len(response_bytes) != expected_len:
13        print("Error: Incomplete frame received.")
14        return None
15
16    # Example validation: Check start byte and checksum
17    start_byte = response_bytes[0]
18    message_part = response_bytes[1:-1]
19    checksum_received = response_bytes[-1]
20
21    if start_byte != 0xAA:
22        print("Error: Invalid start byte.")
23        return None
24
25    if calculate_checksum(message_part) != checksum_received:
26        print("Error: Checksum mismatch.")
27        return None
28
29    print("Frame is valid.")
30    return list(message_part) # Return the useful data
```

Figure 47: RS-485 Communication 5

Main Logic

```
1  # --- Main Logic ---
2  if __name__ == "__main__":
3      try:
4          # Example: Send a command to device 1 to set a value
5          device_id = 0x01
6          command = 0x10 # e.g., 'SET_VALUE' command
7          payload = [0x41, 0x42] # e.g., some data 'AB'
8
9          send_custom_frame(device_id, command, payload)
10
11         # Now, try to read the response. Let's assume the device
12         # replies with a 5-byte frame (start, dev_id, status, data, checksum)
13         response_data = receive_custom_frame(expected_len=5)
14
15         if response_data:
16             print(f"Parsed Response Data: {response_data}")
17
18     except KeyboardInterrupt:
19         print("\nProgram terminated by user.")
20     finally:
21         if ser.is_open:
22             ser.close()
23         print("Serial port closed.")
```

Figure 48: RS-485 Communication 6

For detailed RS-485 communication protocol of the gripper, please refer to the user manual.

Modbus RTU Communication Protocol

Introduction

Modbus RTU is a communication protocol that defines a message structure for devices to exchange data. It runs on top of serial communication layers like RS485 or RS232. Think of it this way: RS485 is the *physical mail system* (the wires and electrical signals), while Modbus RTU is the *language written on the letter* (the specific format of the request and response).

Key Concepts of Modbus RTU

1. **Master/Slave Architecture:** The protocol works on a **master/slave** (or more modernly, **client/server**) model.
 - **Master (Client):** The device that initiates communication. It sends requests (queries) to slaves. There's only one active master on the bus at any time. Your computer running Python will be the master.
 - **Slave (Server):** The peripheral device (like a sensor, actuator, or PLC) that listens for requests. It responds to the master but never initiates communication. Each slave on the bus must have a unique ID, typically from 1 to 247.

2. Data Model (Register Types): Modbus organizes data into four basic types, stored in a 16-bit register address space:

Register Type	Access	Size	Common Use
Coils	Read/Write	1-bit	Outputs, like turning a relay or a light on/off.
Discrete Inputs	Read-Only	1-bit	Inputs, like reading the status of a digital switch.
Holding Registers	Read/Write	16-bit	Configuration data, like setting a temperature setpoint.
Input Registers	Read-Only	16-bit	Measurement data, like reading a temperature value.

3. The Modbus RTU Frame: This is what the library will build for you automatically. A message, or "frame," has a standard structure:

Slave ID (1 byte)	Function Code (1 byte)	Data (N bytes)	CRC (2 bytes)
The unique ID of the target device (1-247).	The command to execute (e.g., 0x03 for "Read Holding Registers").	The data needed for the command (e.g., register address, number of registers, data to write).	A 16-bit Cyclic Redundancy Check for error detection. This is the main advantage over a simple checksum.

Using Modbus RTU with Python

The easiest way to use Modbus RTU in Python is with a dedicated library. *minimalmodbus* is excellent for its simplicity and ease of use.

Import Packages and Setup Configuration

```
1 import minimalmodbus
2 import serial
3 import time
4
5 # --- Configuration ---
6 # ⚠ Update these values to match your setup
7 SERIAL_PORT = '/dev/ttyUSB0' # or 'COM3', 'COM4', etc. on Windows
8 GRIPPER_ID = 1               # The slave address of your gripper
9 BAUD_RATE = 115200           # Default baud rate is 115200 [cite: 455]
```

Figure 49: Modbus RTU Communication 1

Establish Connection

```
1  # --- Configuration ---
2  # Update these values to match your setup
3  SERIAL_PORT = '/dev/ttyUSB0' # or 'COM3', 'COM4', etc. on Windows
4  GRIPPER_ID = 1               # The slave address of your gripper
5  BAUD_RATE = 115200          # Default baud rate is 115200
6
7  # --- Initialize the Gripper (Modbus Master) ---
8  instrument = minimalmodbus.Instrument(SERIAL_PORT, GRIPPER_ID)
9
10 # --- Configure Serial Port Settings ---
11 instrument.serial.baudrate = BAUD_RATE
12 instrument.serial.bytesize = 8
13 instrument.serial.parity = serial.PARITY_NONE
14 instrument.serial.stopbits = 1
15 instrument.serial.timeout = 0.5 # seconds
16 instrument.mode = minimalmodbus.MODE_RTU
```

Figure 50: Modbus RTU Communication 2\

Set Speed and Force

```
1  # --- 1. Set Speed and Force ---
2  # According to the manual, speed and force are set first, but the gripper
3  # does not move until the opening is set.
4
5  # Set speed to 500 (Range: 10-1000)
6  # Register Address: 11
7  target_speed = 500
8  instrument.write_register(registeraddress=11, value=target_speed, functioncode=6)
9  print(f"Speed set to: {target_speed}")
10
11 # Set force to 200 (Range: 100-1000)
12 # Register Address: 12
13 target_force = 200
14 instrument.write_register(registeraddress=12, value=target_force, functioncode=6)
15 print(f"Force set to: {target_force}")
16
17 time.sleep(0.1) # Short delay after setting parameters
```

Figure 51: Modbus RTU Communication 3

Read Status

```
1  # --- 3. Read Status and Actual Opening ---
2  print("\nReading gripper status...")
3
4  # Read the actual opening of the gripper (Read-Only)
5  # Register Address: 61
6  actual_opening = instrument.read_register(registeraddress=61,functioncode=3)
7  print(f" Actual Opening: {actual_opening}")
8
9  # Read the overall status code (Read-Only)
10 # Register Address: 65
11 status_code = instrument.read_register(registeraddress=65,functioncode=3)
12 status_map = {
13     1: "Unclenched to max opening and stopped",
14     2: "Clenched to min opening and stopped",
15     3: "Stopped",
16     4: "Grasping",
17     5: "Unclenching",
18     6: "Grasped an object and stopped"
19 }
20 print(f" Status Code: {status_code} ({status_map.get(status_code, 'Unknown')})")
```

Figure 52: Modbus RTU Communication 4

Fully Open Gripper

```
1  # --- 4. Open the gripper fully ---
2  print("\nMoving gripper to maximum opening (1000)...")
3  instrument.write_register(registeraddress=10,value=1000,functioncode=6)
4  time.sleep(2)
5  actual_opening = instrument.read_register(registeraddress=61,functioncode=3)
6  print(f" Final Actual Opening: {actual_opening}")
```

Figure 53: Modbus RTU Communication 5

For detailed Modbus RTU communication protocol of the gripper, please refer to the user manual.